PRACTICAL – 9

Implementation of prim's algorithm.

Aim:
To implement **Prim's Algorithm** to find the **Minimum Spanning Tree (MST)** of a connected, weighted, undirected graph.

Objective:
To implement **Prim's Algorithm** for finding the **Minimum Spanning Tree (MST)** of a connected, weighted,undirected graph by selecting the minimum-weight edge at each step without formingacycle.

Algorithm:

- Startwithanyvertex, usually vertex 0.
- Mark it as visited.
- Findtheminimum-weight edge connecting a visited vertex to an unvisited vertex.
- Addthatedgetothe MST.
- Markthenewlyselected vertex as visited.
- Repeatuntilallvertices are included in the MST.
- Calculatethetotalweight of the selected edges.

Code:

```
V = 5
graph = [
   [0, 2, 0, 6, 0],
   [2, 0, 3, 8, 5],
   [0, 3, 0, 0, 7],
   [6, 8, 0, 0, 9],
   [0, 5, 7, 9, 0]
]
visited = [False] * V
key = [float('inf')] * V
parent = [-1] * V

key[0] = 0

for count in range(V):
   minimum = float('inf')
```

```
u = -1

    for i in range(V):
        if not visited[i] and key[i] < minimum:
            minimum = key[i]
            u = i

    if u == -1:
        break

    visited[u] = True

    for v in range(V):
        if graph[u][v] != 0 and not visited[v] and graph[u][v] < key[v]:
            key[v] = graph[u][v]
            parent[v] = u

total = 0
print("Edge    Weight")

for i in range(1, V):
    print(parent[i], "-", i, "    ", graph[i][parent[i]])
    total += graph[i][parent[i]]

print("Total weight of MST =", total)
print("EN.No: 92460118708")
```

Output:



Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*O(V^2)*	*Checks vertices to find the minimum edge.*
Average Case	*O(V^2)*	*Selects minimum-weight edges.*
Worst case	*O(V^2)*	*Checks all vertices in each iteration.*

Space Complexity:

Space Complexity	*Description*
O(V^2)	**A 2D matrix is used to store the graph.**